# Resolución — Taller SQL sobre HR 9 — Reportes de gerencia: ROLLUP, CUBE y PIVOT

Bases de Datos I · resolución paso a paso

## Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta, su resultado en HR y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR. Los resultados salen de ejecutarlas ahí.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

## Ejercicio 1 — Nómina por departamento con total general

**Nivel:** Intermedio · **Tablas:** `employees`, `departments`

**Se pide.** Mostrar, por departamento, la cantidad de empleados y la nómina, y al final una fila «TOTAL» con la suma general.

**Mostrar:** departamento, empleados y nómina, de mayor a menor nómina y con la fila TOTAL al final.

**Idea.** `ROLLUP(a)` calcula el agrupamiento normal y agrega una fila con el total general. `GROUPING(a)` vale 1 en esa fila de total y 0 en las demás.

**Pasos.**

1. Se unen `hr.employees` y `hr.departments` y se agrupa con `GROUP BY ROLLUP(d.department_name)`.
2. Además de un grupo por departamento, `ROLLUP` agrega una fila con el total, que trae `NULL` en el nombre del departamento.
3. `GROUPING(d.department_name) = 1` reconoce esa fila y el `CASE` le pone el texto «TOTAL».
4. `ORDER BY GROUPING(...), nomina DESC` deja el total al final y los departamentos de mayor a menor nómina.

```
SELECT CASE WHEN GROUPING(d.department_name) = 1
            THEN 'TOTAL'
            ELSE d.department_name
       END AS departamento,
       COUNT(*) AS empleados,
       SUM(e.salary) AS nomina
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY ROLLUP(d.department_name)
ORDER BY GROUPING(d.department_name), nomina DESC;
```

**Resultado en HR.** 12 filas y 3 columnas; se muestran las primeras 6 filas:

```
DEPARTAMENTO  EMPLEADOS  NOMINA
------------  ---------  ------
Sales                34  304500
Shipping             45  156400
Executive             3   58000
Finance               6   51608
IT                    5   28800
Purchasing            6   24900
```

**Ojo.** La fila de total tiene `NULL` en la columna agrupada, y `GROUPING` es la forma correcta de reconocerla. El total, 684416 de nómina y 106 empleados, no incluye a Kimberely Grant: el `JOIN` interno la deja fuera.

## Ejercicio 2 — Departamento y cargo con subtotales

**Nivel:** Avanzado · **Tablas:** `employees`, `departments`, `jobs`

**Se pide.** Mostrar la nómina por departamento y cargo, con un subtotal por cada departamento y un total general.

**Mostrar:** departamento, cargo, empleados y nómina; cada departamento con sus cargos, luego su subtotal, y al final el total general.

**Idea.** `ROLLUP(a, b)` agrega subtotales por a y un total general, quitando columnas de derecha a izquierda. `GROUPING_ID(a, b)` da un número que dice qué columnas están agregadas en cada fila: 0 detalle, 1 subtotal de a, 3 total general.

**Pasos.**

1. `GROUP BY ROLLUP(d.department_name, j.job_title)` genera el detalle, el subtotal por departamento y el total general.
2. `GROUPING_ID(d.department_name, j.job_title)` es 0 en el detalle, 1 en el subtotal y 3 en el total.
3. Dos `CASE` convierten esos valores en etiquetas legibles.
4. El `ORDER BY` pone cada departamento con sus cargos primero y su subtotal después, y el total general al final.

```
SELECT CASE WHEN GROUPING(d.department_name) = 1
            THEN 'TOTAL GENERAL'
            ELSE d.department_name
       END AS departamento,
       CASE GROUPING_ID(d.department_name, j.job_title)
         WHEN 0 THEN j.job_title
         WHEN 1 THEN 'Subtotal del departamento'
         ELSE 'Total general'
       END AS cargo,
       COUNT(*) AS empleados,
       SUM(e.salary) AS nomina
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.jobs j ON j.job_id = e.job_id
GROUP BY ROLLUP(d.department_name, j.job_title)
ORDER BY GROUPING(d.department_name), d.department_name,
         GROUPING(j.job_title), j.job_title;
```

**Resultado en HR.** 31 filas y 4 columnas; se muestran las primeras 6 filas:

```
DEPARTAMENTO    CARGO                          EMPLEADOS  NOMINA
--------------  -----------------------------  ---------  ------
Accounting      Accounting Manager                     1   12008
Accounting      Public Accountant                      1    8300
Accounting      Subtotal del departamento              2   20308
Administration  Administration Assistant               1    4400
Administration  Subtotal del departamento              1    4400
Executive       Administration Vice President          2   34000
```

**Ojo.** `GROUPING_ID` evita encadenar varios `GROUPING`. Salen 31 filas: 19 combinaciones de departamento y cargo, 11 subtotales y 1 total general. `ROLLUP` no ordena: el orden hay que pedirlo.

## Ejercicio 3 — ROLLUP parcial: subtotales sin total general

**Nivel:** Avanzado · **Tablas:** `employees`, `departments`, `jobs`

**Se pide.** Repetir el reporte anterior pero con subtotales por departamento y sin la fila de total general.

**Mostrar:** departamento, cargo y nómina, con cada departamento seguido de su subtotal.

**Idea.** ROLLUP puede aplicarse solo a una parte del GROUP BY: las columnas que quedan fuera siempre se agrupan, y las de dentro generan los subtotales.

**Pasos.**

1. `GROUP BY d.department_name, ROLLUP(j.job_title)` agrupa siempre por departamento y agrega un subtotal por departamento al quitar el cargo.
2. Como el departamento queda fuera del ROLLUP, nunca se convierte en NULL y no hay total general.
3. `GROUPING(j.job_title) = 1` reconoce los subtotales para ponerles el texto «Subtotal».

```
SELECT d.department_name,
       CASE WHEN GROUPING(j.job_title) = 1
            THEN 'Subtotal'
            ELSE j.job_title
       END AS cargo,
       SUM(e.salary) AS nomina
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.jobs j ON j.job_id = e.job_id
GROUP BY d.department_name, ROLLUP(j.job_title)
ORDER BY d.department_name, GROUPING(j.job_title), j.job_title;
```

**Resultado en HR.** 30 filas y 3 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_NAME  CARGO                          NOMINA
---------------  -----------------------------  ------
Accounting       Accounting Manager              12008
Accounting       Public Accountant                8300
Accounting       Subtotal                        20308
Administration   Administration Assistant         4400
Administration   Subtotal                         4400
Executive        Administration Vice President   34000
```

**Ojo.** Es el ejercicio anterior sin la fila de total general: 30 filas en lugar de 31. Las columnas fuera del ROLLUP no pasan a NULL, por eso `d.department_name` no necesita un CASE.

## Ejercicio 4 — CUBE: departamento por año de contratación

**Nivel:** Avanzado · **Tablas:** `employees`, `departments`

**Se pide.** Para los departamentos Finance, IT y Purchasing y las contrataciones de 2015, 2016 y 2017, mostrar la cantidad de empleados y la nómina por cada combinación de departamento y año, con todos los subtotales posibles.

**Mostrar:** departamento, año, empleados y nómina; con «Todos» donde la columna está agregada.

**Idea.** `CUBE(a, b)` calcula todos los agrupamientos posibles: por a y b juntos, solo por a, solo por b y el total general.

**Pasos.**

1. Una vista en línea filtra los tres departamentos y los tres años y trae el departamento, el año y el salario.
2. `GROUP BY CUBE(x.departamento, x.anio)` genera los cuatro niveles de agrupamiento.
3. GROUPING en cada columna reconoce cuándo está agregada y los CASE la rotulan «Todos».
4. El ORDER BY deja primero las filas por departamento y al final las que agregan el departamento.

```
SELECT CASE WHEN GROUPING(x.departamento) = 1
            THEN 'Todos'
            ELSE x.departamento
       END AS departamento,
       CASE WHEN GROUPING(x.anio) = 1
            THEN 'Todos'
            ELSE TO_CHAR(x.anio)
       END AS anio,
       COUNT(*) AS empleados,
       SUM(x.salary) AS nomina
FROM (SELECT d.department_name AS departamento,
             EXTRACT(YEAR FROM e.hire_date) AS anio,
             e.salary
      FROM hr.employees e
      JOIN hr.departments d ON d.department_id = e.department_id
      WHERE d.department_name IN ('Finance', 'IT', 'Purchasing')
        AND EXTRACT(YEAR FROM e.hire_date) BETWEEN 2015 AND 2017) x
GROUP BY CUBE(x.departamento, x.anio)
ORDER BY GROUPING(x.departamento), x.departamento,
         GROUPING(x.anio), x.anio;
```

**Resultado en HR.** 16 filas y 4 columnas; se muestran las primeras 6 filas:

```
DEPARTAMENTO  ANIO   EMPLEADOS  NOMINA
------------  -----  ---------  ------
Finance       2015           2   15900
Finance       2016           1    7800
Finance       2017           1    6900
Finance       Todos          4   30600
IT            2015           1    4800
IT            2016           2   13800
```

**Ojo.** Con dos columnas CUBE da cuatro niveles de agrupamiento y ROLLUP solo tres: CUBE agrega también los subtotales por año, que ROLLUP no calcula. Salen 16 filas: 9 combinaciones, 3 subtotales por departamento, 3 por año y el total general.

## Ejercicio 5 — GROUPING SETS: tres cortes en una consulta

**Nivel:** Avanzado · **Tablas:** `employees`, `departments`, `jobs`

**Se pide.** En una sola consulta, mostrar la nómina por departamento, la nómina por cargo y el total general.

**Mostrar:** departamento, cargo, empleados y nómina, con los cortes por departamento primero, luego por cargo y al final el total.

**Idea.** `GROUPING SETS` lista exactamente los cortes que se quieren: aquí por departamento, por cargo y el total general. Equivale a unir tres `GROUP BY` con `UNION ALL`, pero escrito como una sola consulta.

**Pasos.**

1. `GROUPING SETS ((d.department_name), (j.job_title), ())` define los tres cortes; el paréntesis vacío es el total general.
2. En cada corte las columnas que no participan salen en `NULL`.
3. `GROUPING_ID(d.department_name, j.job_title)` dice de qué corte viene cada fila: 1, 2 o 3.
4. Se ordena por ese número para agrupar los cortes.

```
SELECT d.department_name,
       j.job_title,
       COUNT(*) AS empleados,
       SUM(e.salary) AS nomina
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.jobs j ON j.job_id = e.job_id
GROUP BY GROUPING SETS ((d.department_name), (j.job_title), ())
ORDER BY GROUPING_ID(d.department_name, j.job_title),
         d.department_name, j.job_title;
```

**Resultado en HR.** 31 filas y 4 columnas; se muestran las primeras 6 filas:

```
DEPARTMENT_NAME  JOB_TITLE  EMPLEADOS  NOMINA
---------------  ---------  ---------  ------
Accounting       (null)             2   20308
Administration   (null)             1    4400
Executive        (null)             3   58000
Finance          (null)             6   51608
Human Resources  (null)             1    6500
IT               (null)             5   28800
```

**Ojo.** Las columnas que no participan en un corte salen en `NULL`: en las filas por departamento `job_title` es `NULL`, y al revés. Salen 31 filas: 11 por departamento, 19 por cargo y 1 de total. `ROLLUP` y `CUBE` son atajos de `GROUPING SETS` para los casos más comunes.

## Ejercicio 6 — El NULL real y el NULL de un subtotal

**Nivel:** Avanzado · **Tablas:** `employees`

**Se pide.** Mostrar la cantidad de empleados y la nómina por código de departamento, incluyendo a la persona que no tiene departamento, y un total general. Las dos filas con código nulo deben distinguirse.

**Mostrar:** departamento («Sin departamento», el código o «TOTAL»), empleados y nómina, con quien no tiene departamento primero y el total al final.

**Idea.** En una consulta con `ROLLUP`, un `NULL` en la columna agrupada puede significar dos cosas: que el dato real es `NULL` o que la fila es un subtotal. `GROUPING` las distingue: vale 1 solo en el subtotal.

**Pasos.**

1. `GROUP BY ROLLUP(e.department_id)` agrupa por código y agrega el total general.
2. Hay dos filas con `department_id` nulo: la de Kimberely Grant, que es un grupo real, y la de total.
3. `CASE GROUPING(e.department_id) WHEN 1 THEN 'TOTAL'` reconoce la de total; la otra se rotula con `NVL`.
4. `ORDER BY GROUPING(...), e.department_id NULLS FIRST` pone primero al grupo sin departamento y al final el total.

```
SELECT CASE GROUPING(e.department_id)
         WHEN 1 THEN 'TOTAL'
         ELSE NVL(TO_CHAR(e.department_id), 'Sin departamento')
       END AS departamento,
       COUNT(*) AS empleados,
       SUM(e.salary) AS nomina
FROM hr.employees e
GROUP BY ROLLUP(e.department_id)
ORDER BY GROUPING(e.department_id), e.department_id NULLS FIRST;
```

**Resultado en HR.** 13 filas y 3 columnas; se muestran las primeras 6 filas:

```
DEPARTAMENTO      EMPLEADOS  NOMINA
----------------  ---------  ------
Sin departamento          1    7000
10                        1    4400
20                        2   19000
30                        6   24900
40                        1    6500
50                       45  156400
```

**Ojo.** Sin `GROUPING` las dos filas se verían iguales, con `NULL` en el departamento: una con 1 empleado y otra con 107. Este total, 107 empleados y 691416 de nómina, sí incluye a Kimberely Grant, porque aquí no hay `JOIN` con departamentos.

## Ejercicio 7 — Subtotales por región, país y ciudad

**Nivel:** Avanzado · **Tablas:** `employees`, `departments`, `locations`, `countries`, `regions`

**Se pide.** Mostrar la cantidad de empleados y la nómina por región, país y ciudad, con un subtotal por país, un subtotal por región y un total general.

**Mostrar:** región, código del país, ciudad, empleados y nómina; cada nivel seguido de su subtotal, con `NULL` en las columnas agregadas.

**Idea.** Con tres columnas, `ROLLUP` produce un subtotal por cada prefijo: región, país y ciudad; región y país; solo región; y el total general.

**Pasos.**

1. Se encadenan las cinco tablas desde `hr.employees` hasta `hr.regions`.
2. `GROUP BY ROLLUP(r.region_name, c.country_id, l.city)` genera los cuatro niveles.
3. El `ORDER BY` usa `GROUPING` de cada columna para que cada detalle quede antes de su subtotal.

```
SELECT r.region_name,
       c.country_id,
       l.city,
       COUNT(*) AS empleados,
       SUM(e.salary) AS nomina
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.locations l ON l.location_id = d.location_id
JOIN hr.countries c ON c.country_id = l.country_id
JOIN hr.regions r ON r.region_id = c.region_id
GROUP BY ROLLUP(r.region_name, c.country_id, l.city)
ORDER BY GROUPING(r.region_name), r.region_name,
         GROUPING(c.country_id), c.country_id,
         GROUPING(l.city), l.city;
```

**Resultado en HR.** 14 filas y 5 columnas; se muestran las primeras 6 filas:

```
REGION_NAME  COUNTRY_ID  CITY                EMPLEADOS  NOMINA
-----------  ----------  ------------------  ---------  ------
Americas     CA          Toronto                     2   19000
Americas     CA          (null)                      2   19000
Americas     US          Seattle                    18  159216
Americas     US          South San Francisco        45  156400
Americas     US          Southlake                   5   28800
Americas     US          (null)                     68  344416
```

**Ojo.** Salen 14 filas: 7 ciudades, 4 subtotales por país, 2 por región y 1 total. Un país con una sola ciudad repite sus números en el subtotal: Canadá y Toronto, por ejemplo, tienen las mismas dos personas y la misma nómina.

## Ejercicio 8 — PIVOT: contrataciones por año en columnas

**Nivel:** Avanzado · **Tablas:** `employees`, `departments`

**Se pide.** Mostrar, para cada departamento, cuántas personas se contrataron en cada año de 2014 a 2018, con un año por columna.

**Mostrar:** departamento y una columna por año (`a2014` a `a2018`) con la cantidad de contratados, ordenados por departamento.

**Idea.** `PIVOT` convierte los valores de una columna en columnas nuevas: `FOR anio IN (2014, 2015, ...)` crea una columna por cada año de la lista y la función de agregación llena cada celda.

**Pasos.**

1. La vista en línea trae el departamento y el año de contratación de cada empleado.
2. `PIVOT (COUNT(*) FOR anio IN (...))` cuenta, para cada departamento, cuántas filas hay en cada año de la lista.
3. Cada año lleva un alias (`AS a2014`) que da nombre a la columna.
4. Las demás columnas de la vista, aquí solo el departamento, forman las filas del resultado.

```
SELECT *
FROM (SELECT d.department_name AS departamento,
             EXTRACT(YEAR FROM e.hire_date) AS anio
      FROM hr.employees e
      JOIN hr.departments d ON d.department_id = e.department_id)
PIVOT (COUNT(*)
       FOR anio IN (2014 AS a2014, 2015 AS a2015, 2016 AS a2016,
                    2017 AS a2017, 2018 AS a2018))
ORDER BY departamento;
```

**Resultado en HR.** 11 filas y 6 columnas; se muestran las primeras 6 filas:

```
DEPARTAMENTO     A2014  A2015  A2016  A2017  A2018
---------------  -----  -----  -----  -----  -----
Accounting           0      0      0      0      0
Administration       0      0      0      0      0
Executive            0      1      0      0      0
Finance              0      2      1      1      0
Human Resources      0      0      0      0      0
IT                   0      1      2      2      0
```

**Ojo.** Solo se cuentan los años de la lista: las contrataciones de 2011, 2012 y 2013 no aparecen en ninguna columna, y por eso todas las celdas suman 92 y no 106. Los valores que se pivotan hay que escribirlos a mano: `PIVOT` no los descubre.

**Variante.** La misma tabla con agregación condicional funciona en cualquier motor, no solo en Oracle:

```
SELECT d.department_name AS departamento,
       COUNT(CASE WHEN EXTRACT(YEAR FROM e.hire_date) = 2014
                  THEN 1 END) AS a2014,
       COUNT(CASE WHEN EXTRACT(YEAR FROM e.hire_date) = 2015
                  THEN 1 END) AS a2015,
       COUNT(CASE WHEN EXTRACT(YEAR FROM e.hire_date) = 2016
                  THEN 1 END) AS a2016,
       COUNT(CASE WHEN EXTRACT(YEAR FROM e.hire_date) = 2017
                  THEN 1 END) AS a2017,
       COUNT(CASE WHEN EXTRACT(YEAR FROM e.hire_date) = 2018
                  THEN 1 END) AS a2018
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY d.department_name
ORDER BY departamento;
```

## Ejercicio 9 — PIVOT con bandas de salario

**Nivel:** Avanzado · **Tablas:** `employees`, `departments`, `locations`, `countries`

**Se pide.** Clasificar los salarios en tres bandas (menos de 5000 es «Bajo», de 5000 a 10000 es «Medio» y más de 10000 es «Alto») y mostrar, para cada país, cuántos empleados hay en cada banda, con una columna por banda.

**Mostrar:** código del país y una columna por banda (`bajo`, `medio`, `alto`) con la cantidad de empleados, ordenados por país.

**Idea.** `PIVOT` trabaja con cualquier columna calculada: primero se clasifica el salario en una banda con `CASE` y luego se pivotan las bandas.

**Pasos.**

1. La vista en línea une `hr.employees` con el país del departamento y calcula la banda con `CASE`.
2. `PIVOT (COUNT(*) FOR banda IN (...))` cuenta empleados por país y por banda.
3. Los valores de la lista llevan alias en minúsculas para nombrar las columnas.

```
SELECT *
FROM (SELECT c.country_id AS pais,
             CASE WHEN e.salary < 5000 THEN 'Bajo'
                  WHEN e.salary <= 10000 THEN 'Medio'
                  ELSE 'Alto'
             END AS banda
      FROM hr.employees e
      JOIN hr.departments d ON d.department_id = e.department_id
      JOIN hr.locations l ON l.location_id = d.location_id
      JOIN hr.countries c ON c.country_id = l.country_id)
PIVOT (COUNT(*)
       FOR banda IN ('Bajo' AS bajo, 'Medio' AS medio, 'Alto' AS alto))
ORDER BY pais;
```

**Resultado en HR.** 4 filas y 4 columnas:

```
PAIS  BAJO  MEDIO  ALTO
----  ----  -----  ----
CA       0      1     1
DE       0      1     0
GB       0     27     8
US      49     13     6
```

**Ojo.** Los valores de la lista del `FOR ... IN` se escriben tal como están en los datos, con las mayúsculas exactas: `'Bajo'` y `'bajo'` serían valores distintos y una banda que no coincide no se cuenta. Las filas suman 106, los empleados que tienen departamento.

## Ejercicio 10 — UNPIVOT: de columnas a filas

**Nivel:** Avanzado · **Tablas:** `employees`

**Se pide.** Para los empleados de ventas (`department_id` 80) contratados en 2018, mostrar cada ingreso en una fila aparte: una fila para el salario y otra para la comisión, con la comisión calculada como salario por `commission_pct`.

**Mostrar:** nombre completo, concepto («Salario» o «Comisión») y monto, ordenados por nombre y concepto.

**Idea.** `UNPIVOT` hace lo contrario de `PIVOT`: convierte columnas en filas. Cada columna listada genera una fila por registro, con el nombre del concepto y el valor.

**Pasos.**

1. La vista en línea p calcula una columna `salario` y una `comision` por empleado.
2. `UNPIVOT (monto FOR concepto IN (salario AS 'Salario', comision AS 'Comisión'))` pasa esas dos columnas a filas.
3. `monto` recibe el valor y `concepto` el nombre del ingreso.
4. Se ordena por nombre y concepto.

```
SELECT u.nombre,
       u.concepto,
       u.monto
FROM (SELECT e.first_name || ' ' || e.last_name AS nombre,
             e.salary AS salario,
             e.salary * NVL(e.commission_pct, 0) AS comision
      FROM hr.employees e
      WHERE e.department_id = 80
        AND e.hire_date >= DATE '2018-01-01') p
UNPIVOT (monto FOR concepto IN (salario AS 'Salario',
                                comision AS 'Comisión')) u
ORDER BY u.nombre, u.concepto;
```

**Resultado en HR.** 14 filas y 3 columnas; se muestran las primeras 6 filas:

```
NOMBRE           CONCEPTO  MONTO
---------------  --------  -----
Amit Banda       Comisión    620
Amit Banda       Salario    6200
Charles Johnson  Comisión    620
Charles Johnson  Salario    6200
David Lee        Comisión    680
David Lee        Salario    6800
```

**Ojo.** Por defecto `UNPIVOT` descarta las filas cuyo valor es `NULL` (`EXCLUDE NULLS`); con `INCLUDE NULLS` las conserva. Con el departamento 60 (IT), que no tiene comisiones, `salary` y `commission_pct` dan 5 filas con la opción por defecto y 10 con `INCLUDE NULLS`.

## Ejercicio 11 — Porcentaje de cada departamento sobre el total

**Nivel:** Avanzado · **Tablas:** `employees`, `departments`

**Se pide.** Mostrar la nómina de cada departamento, su porcentaje sobre la nómina total y una fila «TOTAL», todo en una sola consulta.

**Mostrar:** departamento, nómina y porcentaje con un decimal, de mayor a menor nómina y con la fila TOTAL al final (100.0).

**Idea.** La función de ventana se calcula después del `ROLLUP`, así que también ve la fila de total. `PARTITION BY GROUPING(...)` pone esa fila en su propia ventana, aparte de los departamentos.

**Pasos.**

1. `ROLLUP(d.department_name)` genera un grupo por departamento y la fila de total.
2. `RATIO_TO_REPORT(SUM(e.salary))` da la fracción de cada nómina dentro de su ventana.
3. `OVER (PARTITION BY GROUPING(d.department_name))` hace que los departamentos sumen 100 entre ellos y que el total quede solo.
4. Se multiplica por 100 y se redondea a un decimal.

```
SELECT CASE WHEN GROUPING(d.department_name) = 1
            THEN 'TOTAL'
            ELSE d.department_name
       END AS departamento,
       SUM(e.salary) AS nomina,
       ROUND(100 * RATIO_TO_REPORT(SUM(e.salary))
                   OVER (PARTITION BY GROUPING(d.department_name)), 1)
         AS porcentaje
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
GROUP BY ROLLUP(d.department_name)
ORDER BY GROUPING(d.department_name), nomina DESC;
```

**Resultado en HR.** 12 filas y 3 columnas; se muestran las primeras 6 filas:

```
DEPARTAMENTO  NOMINA  PORCENTAJE
------------  ------  ----------
Sales         304500        44.5
Shipping      156400        22.9
Executive      58000         8.5
Finance        51608         7.5
IT             28800         4.2
Purchasing     24900         3.6
```

**Ojo.** Sin la partición, cada departamento saldría con la mitad de su porcentaje real: `RATIO_TO_REPORT` dividiría entre la suma de los departamentos más el total, que es el doble. Sales, por ejemplo, saldría con 22.2 en lugar de 44.5.

## Ejercicio 12 — Reto: región y país con el porcentaje de cada nivel

**Nivel:** Reto · **Tablas:** `employees`, `departments`, `locations`, `countries`, `regions`

**Se pide.** Mostrar la nómina por región y país, con un subtotal por región y un total general, y el porcentaje que representa cada fila dentro de su nivel: los países entre sí, las regiones entre sí y el total.

**Mostrar:** región, país, nómina y porcentaje del nivel, con cada región seguida de su subtotal y el total al final.

**Idea.** Se combina todo: `ROLLUP` genera los niveles, `GROUPING_ID` los identifica y `RATIO_TO_REPORT` con `PARTITION BY GROUPING_ID` reparte el 100 % dentro de cada nivel por separado.

**Pasos.**

1. `ROLLUP(r.region_name, c.country_id)` genera los países, el subtotal por región y el total.
2. `GROUPING_ID(r.region_name, c.country_id)` vale 0 en los países, 1 en las regiones y 3 en el total.
3. Dos `CASE` rotulan las filas a partir de ese valor.
4. `PARTITION BY GROUPING_ID(...)` hace que cada nivel sume 100 por su cuenta.

```
SELECT CASE GROUPING_ID(r.region_name, c.country_id)
         WHEN 3 THEN 'TOTAL'
         ELSE r.region_name
       END AS region,
       CASE GROUPING_ID(r.region_name, c.country_id)
         WHEN 0 THEN c.country_id
         WHEN 1 THEN 'Subtotal de la región'
         ELSE '-'
       END AS pais,
       SUM(e.salary) AS nomina,
       ROUND(100 * RATIO_TO_REPORT(SUM(e.salary))
                   OVER (PARTITION BY GROUPING_ID(r.region_name,
                                                  c.country_id)), 1)
         AS pct_del_nivel
FROM hr.employees e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.locations l ON l.location_id = d.location_id
JOIN hr.countries c ON c.country_id = l.country_id
JOIN hr.regions r ON r.region_id = c.region_id
GROUP BY ROLLUP(r.region_name, c.country_id)
ORDER BY GROUPING(r.region_name), r.region_name,
         GROUPING(c.country_id), c.country_id;
```

**Resultado en HR.** 7 filas y 4 columnas:

```
REGION    PAIS                  NOMINA  PCT_DEL_NIVEL
--------  --------------------  ------  -------------
Americas  CA                     19000            2.8
Americas  US                    344416           50.3
Americas  Subtotal de la región 363416           53.1
Europe    DE                     10000            1.5
Europe    GB                    311000           45.4
Europe    Subtotal de la región 321000           46.9
TOTAL     -                     684416            100
```

**Ojo.** Cada nivel suma 100 por separado: los cuatro países suman 100 entre ellos, las dos regiones suman 100 entre ellas y el total es 100. El porcentaje de un país es sobre la nómina de todos los países, no sobre la de su región.